INTERVIEW SUBMISSION

GPMS Backend Engineering Deep Dive

Implementation-level analysis of architecture, domain logic, security controls, and delivery quality in the NestJS + Prisma API.

Project	GPMS (Garment Production Management System) - API Service
Stack	NestJS 11, Prisma 7, PostgreSQL, JWT, Swagger, Zod env validation
Service Purpose	Factory-scoped production operations backend across planning, inventory, cutting, sewing, and QC
Base Prefix	`/api`
Docs Endpoint	`/api/docs`
Reference Date	March 5, 2026
Prepared For	Backend engineering interview review

> This report is built from the current codebase, not from assumptions. It focuses on architecture decisions, business invariants, correctness boundaries, and operational maturity.

1. Executive Summary

GPMS API is a multi-factory operations backend. It is designed around strict factory scoping, explicit workflow transitions, and transactional consistency for production-critical actions.

The core strength is not just CRUD coverage. The system enforces domain invariants across order lifecycle, planning, cutting, sewing, and QC before permitting final shipment/closure progression.

What demonstrates backend depth here

- Global guard chain for authentication, role checks, and factory-level tenancy enforcement.
- Workflow-heavy service logic with precondition validation before state transitions.
- Prisma transaction usage for multi-step updates that must remain atomic.
- DTO + validation constraints at input boundary plus additional semantic checks in services.
- Operational artifacts: migrations, seed strategy, deploy script, Swagger, and health endpoint.

Codebase scope snapshot

13 controllers | 13 services | 21 Prisma models | 4 enums | 38 route handlers

High-level domain chain supported

```
Master Data -> Purchase Order -> Material Requirement -> Inventory -> Plan Assignment -> Cutting Bundles -> Sewing Output -> QC -> PO Shipment/Closure
```

Domain	Main Controller	Backend Responsibility
Auth and Session	`auth.controller.ts`	Login/logout/me, token issuance, cookie strategy
Master Data	`buyers`, `materials`, `styles`	Factory-scoped reference entities and BOM ownership
Orders	`pos.controller.ts`	PO lifecycle, status movement, requirement derivation
Planning	`plans.controller.ts`	Plan creation and PO-item line assignment
Execution	`cutting`, `sewing`, `qc`	Bundle flow, output capture, inspection state

Domain	Main Controller	Backend Responsibility
Reporting	`reports.controller.ts`	Plan vs actual aggregation by date and line

2. Application Architecture

2.1 Bootstrap pipeline

- CORS with explicit allowlist plus credential support.
- Helmet enabled (with non-production cross-origin resource policy relaxation).
- Cookie parser enabled for JWT cookie extraction.
- Global prefix `/api`.
- Validation pipe with `whitelist`, `transform`, `forbidNonWhitelisted`.
- Swagger served at `/api/docs` with cookie auth metadata.

2.2 Module composition

`AppModule` imports:

- `AuthModule`, `BuyersModule`, `CuttingModule`, `InventoryModule`, `MaterialsModule`, `PlansModule`, `PurchaseOrdersModule`, `QcModule`, `SewingModule`, `StylesModule`, `PrismaModule`.
- Global config validation via Zod schema.
- Global throttler: 120 requests per 60 seconds.

2.3 Guard chain and request contract

Guards are applied globally in this order:

1. `ThrottlerGuard`
2. `JwtAuthGuard`
3. `RolesGuard`
4. `FactoryGuard`

This means all protected requests must authenticate first, then optionally satisfy role constraints, then satisfy factory access with `x-factory-id`.

2.4 Tenancy and context resolution

- `FactoryGuard` requires `x-factory-id` unless route is public or explicitly skips factory guard.
- Header is UUID-validated.
- Membership enforced through `UserFactoryAccess` lookup.
- Resolved factory id is attached to request and used by services.

> Strong design choice: tenancy is enforced in one place globally, not repeated ad hoc per controller.

3. Security, Auth, and Session Design

3.1 Authentication flow

1. `POST /api/auth/login` validates credentials with bcrypt.
2. JWT payload includes user identity and role.
3. Token returned in secure cookie `gpms_access`.
4. `GET /api/auth/me` returns profile and accessible factories.
5. `POST /api/auth/logout` clears auth cookie.

3.2 Token extraction strategy

JWT strategy accepts token from two sources:

- Cookie: `gpms_access`
- Bearer token: `Authorization: Bearer ...`

This dual-path strategy improves interoperability with browser and non-browser clients.

3.3 Cookie policy

Cookie options are strict:

- `httpOnly: true`
- `sameSite: "none"`
- `secure: true`
- 7-day max age

3.4 Security strengths and caveats

Area	Current Implementation	Observation
Input validation	Global validation pipe + DTO decorators	Strong baseline for request hygiene
Auth/session	JWT cookie + bearer fallback	Good compatibility model
Rate limit	Global throttler	Useful coarse protection layer
Role controls	`@Roles` decorator infrastructure exists	No active `@Roles(...)` usage found in controllers, so role guard currently does not enforce endpoint-level role restrictions

> Interview discussion point: role primitives exist, but policy is mostly enforced in business logic (for example PO status progression for non-admin) rather than endpoint annotations. This is valid, but should be explicit in security design docs.

4. Data Model and Persistence Strategy

4.1 Prisma model design

The schema models a realistic garment operations graph:

- Access/tenancy: `User`, `Factory`, `UserFactoryAccess`
- Commercial: `Buyer`, `PurchaseOrder`, `POItem`, `Style`
- Materials: `Material`, `BOMHeader`, `BOMItem`, `InventoryTxn`
- Execution: `ProductionPlan`, `PlanLine`, `DailyTarget`, `CuttingBatch`, `Bundle`, `SewingHourlyOutput`, `QCInspection`, `QCDefect`
- Governance: `AuditLog`

4.2 Enum-backed state systems

- `RoleName` for user responsibility classes.
- `POStatus` for lifecycle progression.
- `InventoryTxnType` for stock movement semantics.
- `InspectionType` for QC stage typing.

4.3 Key relational constraints

- `@@unique([factoryId, poNo])` prevents duplicate PO numbers per factory.
- `@@unique([factoryId, styleNo])` prevents duplicate styles per factory.
- `@@unique([factoryId, bundleCode])` enforces bundle code uniqueness.
- `@@unique([planLineId, date])` enforces one target per plan line date.
- `@@unique([userId, factoryId])` for access relation integrity.

4.4 Migration and seed posture

- Migrations present: `init` and `add_audit_log_table`.
- Seed creates demo factories, admin user, and full process-ready sample data chain.
- Seed also includes schema-drift-tolerant audit log behavior detection.

> Seed depth is strong for demos and integration checks because it includes linked records across PO, planning, cutting, sewing, and QC rather than only master data.

5. Core Domain Workflows and Business Rules

5.1 Purchase order lifecycle correctness

`PurchaseOrdersService` enforces a strict transition model:

- Items can be added only while status is `DRAFT`.
- Confirm requires at least one item.
- Non-admin users can only move to the immediate next status.
- Move to `SHIPPED` or `CLOSED` requires readiness checks:
- all PO items have bundles
- bundled quantity covers ordered quantity
- every bundle has latest passing QC inspection

5.2 Planning constraints

`PlansService.assign` enforces:

- Assignment dates inside plan window.
- Daily target dates inside assignment window and plan window.
- No duplicate daily target dates in one assignment payload.
- PO must be active production candidate (not draft, shipped, or closed).
- Non-admin cannot reassign a PO item that already has an assignment.
- Automatic transition of PO from `CONFIRMED` to `IN_PRODUCTION` inside transaction.

5.3 Inventory control

- `receive` requires `refType = GRN`.
- `issueToCutting` requires `refType = ISSUE_CUTTING`.
- Issue path checks available stock before transaction write.
- Writes audit log entries with metadata where table exists.

5.4 Cutting, sewing, and QC state progression

- Cutting batch requires valid PO item in same factory.
- Bundle code generation is deterministic and sequence-based per batch/factory.
- Sewing output auto-updates bundle status to `SEWN` when cumulative output reaches bundle qty.
- QC inspection updates bundle status to `QC_PASS` or `QC_FAIL`.
- Passing QC rejects defect payload entries to preserve semantics.

5.5 Reporting

- `/reports/plan-vs-actual` merges plan targets with grouped sewing output by line for a date.
- `/qc/summary` computes pass rate, top defects, and DHU-style estimate for selected date.

6. Transaction and Consistency Patterns

6.1 Where transactions are used

Service Method	Transaction Purpose	Consistency Benefit
`styles.upsertBom`	Upsert BOM header, replace BOM items atomically	No partial BOM replacement
`styles.remove`	Delete BOM items/header then style	No orphan BOM data
`plans.assign`	Optional PO status update + plan line + daily targets	Assignment and targets stay coherent
`cutting.createBundles`	Validate duplicates then bulk bundle creation	Prevents partial bundle generation collisions
`inventory.receive / issue`	Create txn + audit log	Ledger/audit coupling
`sewing.createHourlyOutput`	Write output and conditionally update bundle status	Bundle progress reflects output atomically
`qc.inspect`	Create inspection/defects and update bundle status	Inspection result and bundle state stay synced

6.2 Idempotency and duplicate handling notes

- Uniqueness constraints backstop duplicate creation attempts.
- Service-level conflict checks provide user-friendly errors before DB errors where possible.
- Some state-changing endpoints are not idempotent by design (for example hourly output posting).

6.3 Observability posture

- Health endpoint exists: `GET /api/health`.
- Audit log table supports inventory traceability metadata.
- Graceful shutdown hooks enabled at application bootstrap.

> Future maturity opportunity: structured request/response logging and correlation IDs would significantly improve production debugging and root-cause speed.

7. Route and API Surface Summary

7.1 Public and auth routes

- `GET /api`
- `GET /api/health`
- `POST /api/auth/login`
- `POST /api/auth/logout`
- `GET /api/auth/me`

7.2 Factory-scoped route groups

- `/api/buyers` - list/create/update/delete
- `/api/materials` - list/create
- `/api/styles` - style CRUD + BOM upsert/get
- `/api/pos` - PO create, add-item, confirm, status, detail, material requirement
- `/api/inventory` - receive, issue-to-cutting, stock
- `/api/plans` - create, assign, detail
- `/api/reports` - plan-vs-actual
- `/api/cutting` - batch create, bundle create, batch detail, bundle list
- `/api/sewing` - hourly-output, line-status
- `/api/qc` - inspect, summary

7.3 Non-obvious wiring detail

Reports are currently provided through `PlansModule` (controller/service imported there) instead of a dedicated reports module. This works, but a dedicated `ReportsModule` would improve modular clarity.

8. Build, Test, and Quality Evidence

8.1 Build evidence

Command: `pnpm build`

Build passed | Prisma client generation passed

8.2 Test evidence

Command: `pnpm test`

1/1 unit suite passed 1/1 tests passed

Command: `pnpm test:e2e`

e2e failed Missing required env in test context: JWT_SECRET and FRONTEND_URL

> This failure is useful signal: env schema validation is correctly strict, and test harness needs explicit test env provisioning.

9. Risks, Gaps, and Hardening Plan

9.1 Current strengths

- Strong tenant isolation via global factory guard.
- Meaningful business-rule enforcement in lifecycle transitions.
- Good transactional boundaries across multi-step writes.
- Clear DTO validation and consistent error signaling.
- Practical seed data for realistic integration flow demonstration.

9.2 Current gaps

- Role annotation policy is not actively applied at route level.
- e2e setup is incomplete for env-dependent module bootstrap.
- No structured request logging / tracing middleware.
- Limited automated test breadth beyond starter unit test.
- Cookie strategy is secure-first; local HTTP clients may rely on bearer fallback in dev.

9.3 Recommended next milestones

1. Introduce dedicated test env bootstrap for e2e (`.env.test` with required keys).
2. Add role policies with explicit `@Roles(...)` on sensitive write routes.
3. Expand test matrix: critical transitions (confirm -> in_production -> shipped -> closed) and inventory insufficiency scenarios.
4. Add centralized exception filter for normalized error envelopes.
5. Add structured logging with request id correlation and basic latency metrics.
6. Extract reports into dedicated module for cleaner boundaries.

10. Closing Assessment

The backend demonstrates strong practical engineering for operations software: explicit tenancy controls, domain-aware transition rules, and transactional consistency where it matters. The architecture is production-oriented and extensible.

To reach senior-level production maturity, the highest leverage next steps are test depth expansion, route-level role policy clarity, and observability instrumentation.

End of report.